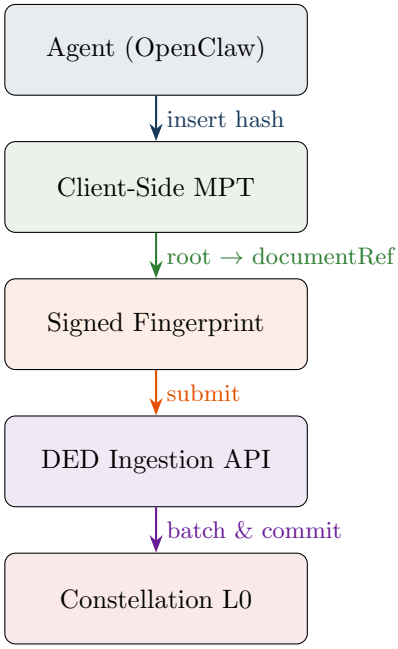
DED Agent WorldState

Verifiable Internal State for Autonomous Agents

Technical Proposal — v0.1

February 13, 2026



Status:	Draft
Authors:	Constellation Network Engineering
Package:	`ded-agent-worldstate` (npm)
Dependencies:	`@constellation-network/ded-sdk`, `@constellation-network/metagraph-sdk`

Contents

1 Motivation **2**

2 Use Cases **2**
2.1 AI Reasoning Audit Trail . . . 2
2.2 Automated Compliance Monitoring . . . 2
2.3 Multi-Step Workflow Orchestration . . . 2

3 System Architecture **3**
3.1 Architecture Diagram . . . 3

4 Data Model **3**
4.1 Extended FingerprintValue . . . 3
4.2 Data Structure Diagram . . . 4
4.3 Evidence Path Construction . . . 4
4.4 Chain Linking Protocol . . . 5

5 State Update Lifecycle **5**

6 Stream State Machine **6**

7 Checkpoint-Based Recovery **6**
7.1 Design Constraint . . . 6
7.2 Checkpoint Model . . . 7
7.3 Recovery Procedure . . . 7
7.4 Checkpoint Cost . . . 7

8 Verification Flow **7**

9 API Surface **8**

10 OpenClaw Integration Example **9**

11 Cost Analysis **10**

12 Threat Model **11**
12.1 Security Properties . . . 11
12.2 Explicit Non-Goals . . . 11

13 Vision: Agent-to-Agent Trust **12**

14 Implementation Roadmap **12**

Appendix A: Server-Side Alignment **13**

Appendix B: MPT Hashing Specification **13**

1 Motivation

Autonomous AI agents are increasingly deployed to make decisions, execute workflows, and interact with external systems on behalf of their operators. As agents gain autonomy, a critical question emerges: **how can an agent prove what it knew, decided, or did at any point in time?**

Today, agent state is ephemeral. Conversation logs, decision records, and internal computations exist only in the agent's local memory or in opaque platform logs. There is no cryptographic guarantee that the agent's reported history is complete, unmodified, or temporally ordered. This creates trust gaps in three domains:

1. **Accountability** — An agent that makes financial, legal, or safety-critical decisions cannot prove the reasoning chain that led to those decisions.
2. **Compliance** — Regulatory frameworks increasingly require auditable records of automated decision-making (e.g., EU AI Act, SEC algorithmic trading rules).
3. **Inter-Agent Trust** — When agents collaborate, there is no mechanism for one agent to verify another's claimed state without direct access to its internals.

The **Digital Evidence Depository (DED)** provides a production-grade notarization engine backed by the Constellation Network's Hypergraph. By combining the DED's fingerprint submission pipeline with a client-side **Merkle Patricia Trie (MPT)**, we can give agents a *verifiable internal world state* — a tamper-evident, temporally ordered, cryptographically committed record of every state transition the agent makes.

2 Use Cases

2.1 AI Reasoning Audit Trail

An AI agent processing insurance claims notarizes each decision point: document receipt, risk assessment, approval/denial, and payout authorization. Each decision is hashed and inserted into the agent's world state trie. If a decision is later disputed, the agent can produce an MPT inclusion proof demonstrating that the decision existed in its state at a specific ordinal, with a corresponding on-chain commitment timestamp.

2.2 Automated Compliance Monitoring

A compliance agent monitors data streams for regulatory violations. Each detected event (or clean-check confirmation) is recorded as a state entry. The chain of fingerprints provides a tamper-evident log that auditors can independently verify against the Constellation L0 without requiring access to the agent's infrastructure.

2.3 Multi-Step Workflow Orchestration

A workflow agent orchestrates a multi-party approval process (e.g., contract signing, supply chain handoffs). Each state transition — initiation, approval, counter-signature, finalization — is committed to the world state. The ordered chain of MPT roots provides a verifiable timeline of the workflow's progression.

3 System Architecture

The system operates as a **two-level commitment architecture**:

1. **Client-Level**: The agent maintains a per-stream MPT. Each state update inserts a hash into the trie, producing a new root. This root becomes the `documentRef` of a signed fingerprint.
2. **Platform-Level**: The DED server batches fingerprints into its own server-side MPT, whose root is committed to the Constellation L0 blockchain via a `MetagraphBatchMessage`.

3.1 Architecture Diagram

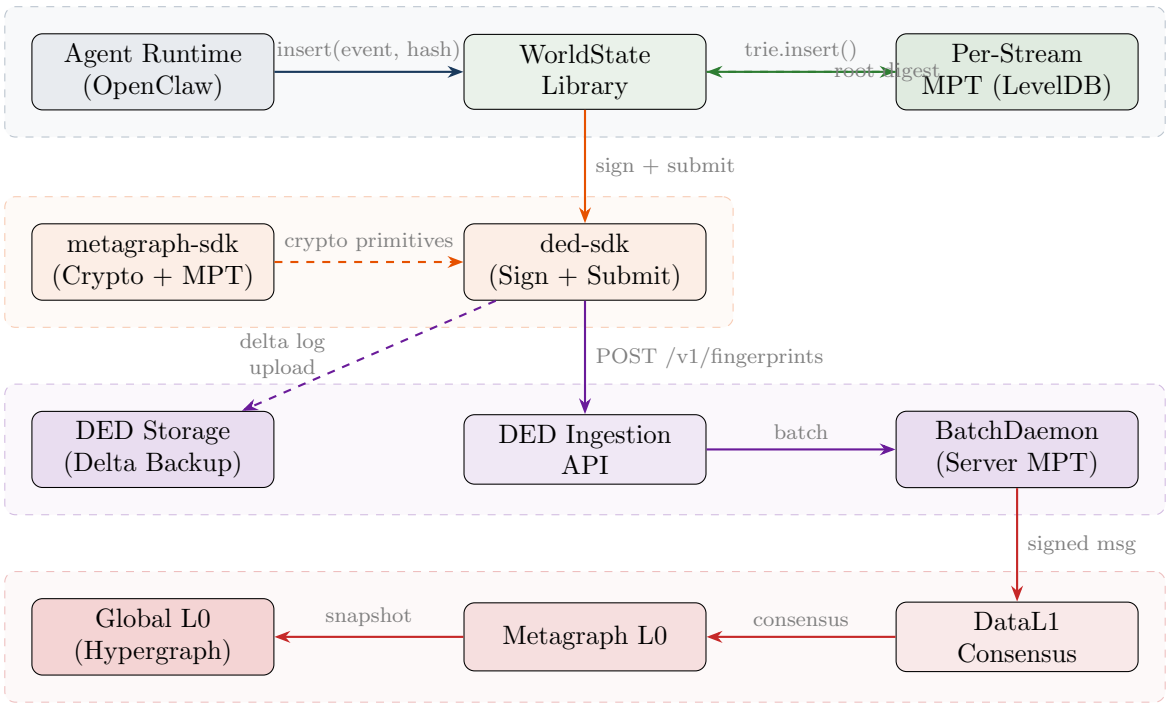


Figure 1: Four-layer system architecture. The agent inserts state hashes into a local MPT, commits roots as fingerprints via the DED SDK, which are batched and committed on-chain.

4 Data Model

4.1 Extended FingerprintValue

The `FingerprintValue` protobuf message is extended with two optional fields:

```
message FingerprintValue {
  string org_id = 1;
  string tenant_id = 2;
  string event_id = 3;
  optional string signer_id = 4;
  string document_id = 5;
  string document_ref = 6;
  Timestamp timestamp = 7;
  uint32 version = 8;

  // NEW: Stream routing fields
  optional string stream_id = 9;   // UUID v4
```

```
  optional string app_id = 10;      // UUID v4
}
```

- `stream_id` and `app_id` are **optional** with a default of the zero UUID (`00000000-0000-0000-0000-000000` when omitted.
- This preserves **backwards compatibility** — existing clients that do not set these fields continue to work unchanged.
- The SDK transparently applies the default when constructing fingerprint values.

4.2 Data Structure Diagram

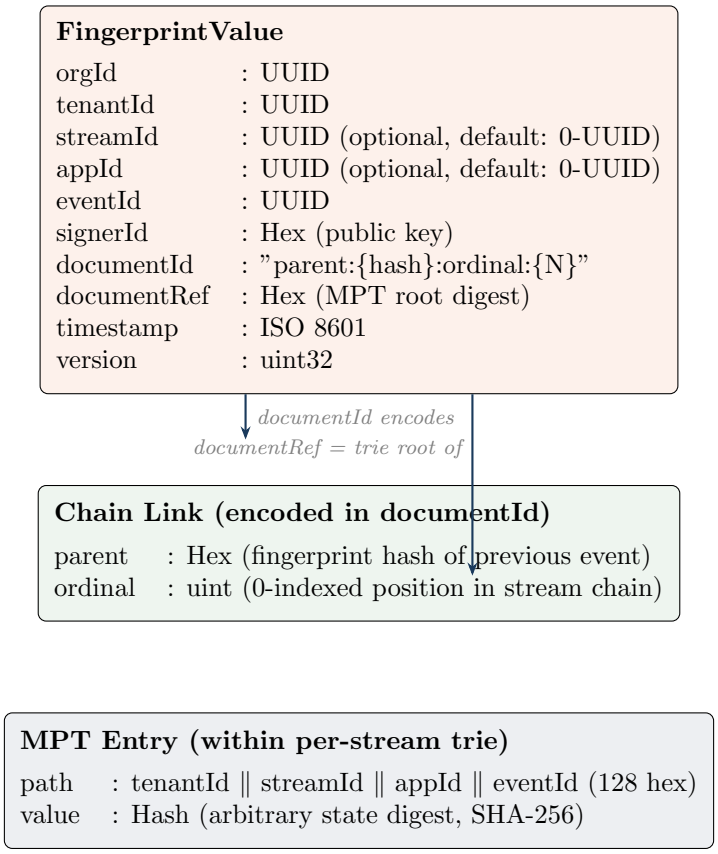


Figure 2: Relationship between FingerprintValue fields, chain linking, and MPT entries.

4.3 Evidence Path Construction

The MPT key path is constructed by concatenating four UUIDs as raw bytes, then hex-encoding:

$$\text{path} = \text{hex}(\text{tenantId}_{16\text{B}} \| \text{streamId}_{16\text{B}} \| \text{appId}_{16\text{B}} \| \text{eventId}_{16\text{B}}) = 128 \text{ hex chars}$$

This matches the server-side `PathEvidenceCalculator` format, ensuring that the client-side and server-side MPT paths are structurally aligned. The SDK enforces this invariant; the server treats `documentRef` as opaque.

4.4 Chain Linking Protocol

Each fingerprint in a stream carries a `documentId` that encodes its position in the revision chain:

documentId Format

```
parent:{fingerprint_hash}:ordinal:{N}
```

- **Genesis event**: `parent:genesis:ordinal:0`
- **Subsequent events**: `parent:{sha256_hash}:ordinal:{N+1}`
- The `fingerprint_hash` is the authoritative hash returned by the DED API upon successful submission of the previous event.
- Chain integrity: the agent waits for the API response (synchronous mode) before constructing the next link.

5 State Update Lifecycle

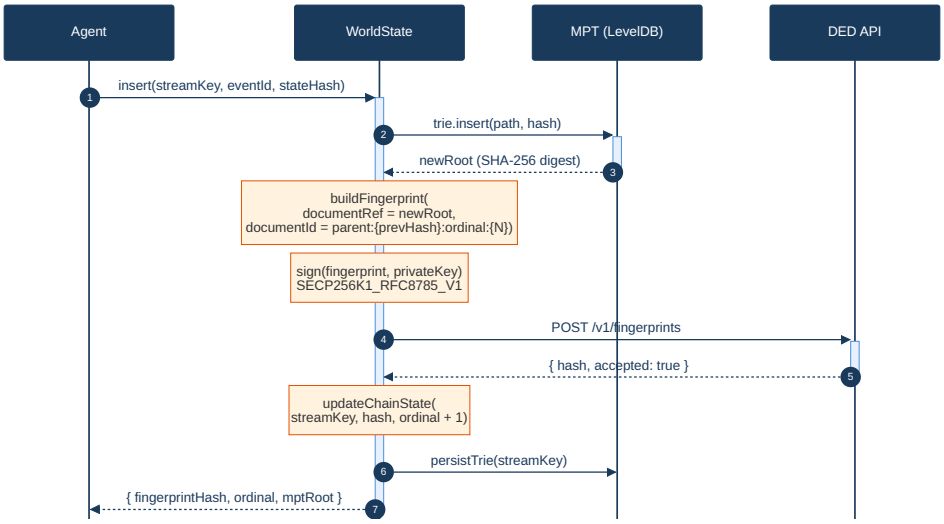


Figure 3: Sequence diagram for a single state update.

The complete lifecycle of a single state insertion (Figure 3):

1. **Agent calls** `worldState.insert(streamKey, eventId, stateHash)`
2. **WorldState** constructs the 128-hex-char MPT path from the stream key components and eventId
3. **MPT.insert** adds the `(path, stateHash)` entry to the per-stream trie and computes the new root digest
4. **WorldState** constructs a `FingerprintValue` with `documentRef` = new MPT root, `documentId` = `parent:{prevHash}:ordinal:{N}`, and `streamId/appId/eventId` from the stream key
5. **Sign** the fingerprint using SECP256K1_RFC8785_V1 with the agent's private key
6. **Submit** to DED Ingestion API; **block** until response
7. **On success**: update local chain state (new parent hash, increment ordinal), persist trie to LevelDB
8. **Return** the fingerprint hash to the agent

6 Stream State Machine

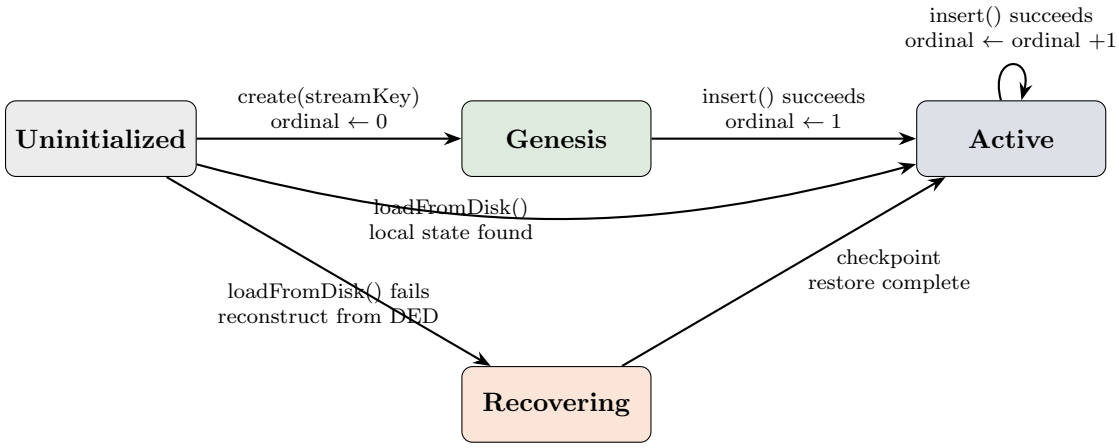


Figure 4: Stream lifecycle state machine. Streams are always open once active. Recovery restores from the latest checkpoint and replays subsequent entries.

- **Uninitialized**: Stream has not been created or loaded.
- **Genesis**: First event pending. `documentId` will be `parent:genesis:ordinal:0`.
- **Active**: Stream is accepting insertions. Each successful insertion increments the ordinal and updates the parent hash.
- **Recovering**: Local state was lost. The library downloads the latest checkpoint from DED Storage and replays any subsequent entries to reconstruct the trie.
- Streams are **always open** — there is no sealed/closed state.

7 Checkpoint-Based Recovery

7.1 Design Constraint

The DED storage system enforces that when a fingerprint includes a document payload, the `documentRef` must equal the SHA-256 hash of that payload. Since WorldState fingerprints use `documentRef` for the MPT root digest, delta entries *cannot* be attached as payloads to the same fingerprint submission. Instead, trie backups are performed as separate **checkpoint jobs**.

7.2 Checkpoint Model

The agent periodically exports a full snapshot of the trie's entries as a separate fingerprint submission dedicated to backup:

```
interface TrieCheckpoint {
  streamKey: StreamKey;        // (tenantId, streamId, appId)
  entries: Record<string, string>;  // path -> hash (all trie entries)
  asOfOrdinal: number;        // Chain ordinal at time of checkpoint
  asOfMptRoot: string;        // MPT root at time of checkpoint
}
```

Checkpoint submission:

- The serialized checkpoint is uploaded to DED Storage as a document payload.
- Its `documentRef` = SHA-256 of the serialized checkpoint (satisfying the storage constraint).
- A dedicated `documentId` format distinguishes checkpoints from state entries: `checkpoint:{streamId}:or`
- Checkpoints use their own stream (e.g., `appId = checkpoint-app-uuid`) to avoid polluting the primary state chain.

7.3 Recovery Procedure

1. Query DED for the latest checkpoint fingerprint for this stream.
2. Download the checkpoint payload from DED Storage.
3. Deserialize the `TrieCheckpoint` and rebuild the MPT from its entries.
4. Query the primary stream chain for any fingerprints with ordinal > `asOfOrdinal`.
5. For each subsequent fingerprint, the agent needs the original `(path, stateHash)` to replay. If objects are stored externally (application database, S3, etc.), re-derive the entry and insert into the trie.
6. Verify the reconstructed trie root matches the latest chain's `documentRef`.

7.4 Checkpoint Cost

A stream with 10,000 entries produces a checkpoint of ∼2 MB (128-char path + 64-char hash per entry). At 1 credit per 100 KB, this costs ∼20 credits ($0.20) per checkpoint plus 2 credits for the fingerprint write. Checkpointing frequency is configurable — e.g., every 1,000 insertions or daily.

8 Verification Flow

An agent can verify that a specific event exists in its committed world state through a three-layer proof chain:

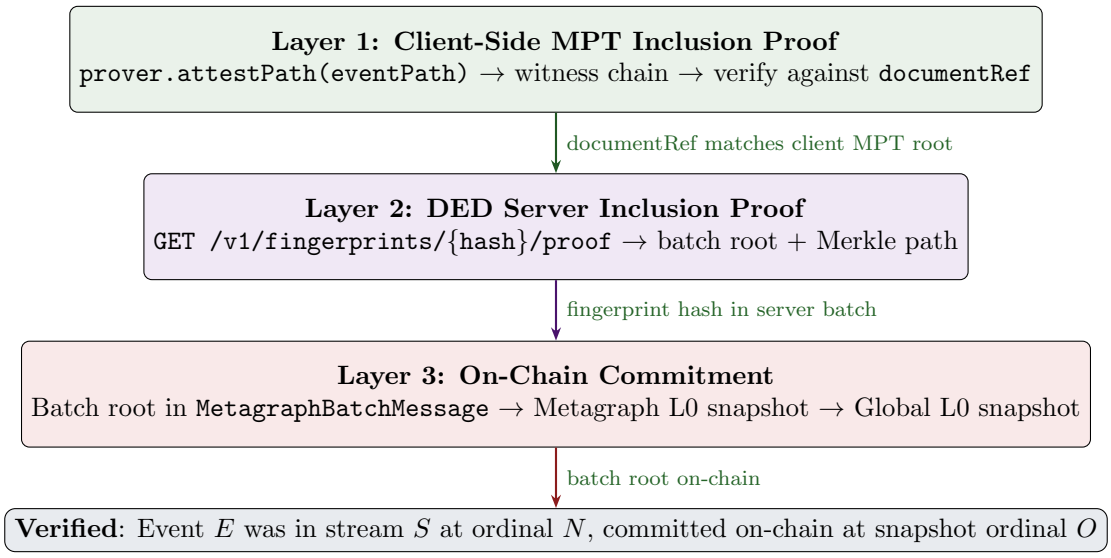


Figure 5: Three-layer verification flow. Each layer's output anchors the layer above it.

Key property: Verification at Layer 1 requires the client's trie (or a reconstructed copy). Layers 2 and 3 are publicly verifiable using only the fingerprint hash and the DED/Constellation public APIs.

9 API Surface

```
import { WorldState } from 'ded-agent-worldstate';

// --- Initialization ---
const ws = await WorldState.create({
  privateKey: '...',           // SECP256K1 private key (hex)
  orgId: 'org-uuid',
  baseUrl: 'https://api.ded.example.com',
  apiKey: 'your-api-key',
  storagePath: './worldstate', // LevelDB directory
});

// --- Define a stream ---
const stream = ws.stream({
  tenantId: 'tenant-uuid',
  streamId: 'stream-uuid',
  appId: 'app-uuid',
});

// --- Insert state ---
const eventId = crypto.randomUUID();
const stateHash = sha256(canonicalize(myStateObject));
const result = await stream.insert(eventId, stateHash);
// result: { fingerprintHash, ordinal, mptRoot }

// --- Verify inclusion ---
const proof = await stream.prove(eventId);
// proof: { clientProof: MerklePatriciaInclusionProof, fingerprintHash }

const isValid = await stream.verify(proof);
// isValid: true

// --- Inspect state ---
const info = await stream.info();
// info: { ordinal, latestRoot, latestFingerprintHash, entryCount }

// --- Checkpoint (backup) ---
await stream.checkpoint();
// Serializes trie entries, uploads to DED Storage

// --- Recovery (from checkpoint) ---
const recovered = await stream.recover();
// Downloads latest checkpoint, replays subsequent entries
```

10 OpenClaw Integration Example

The following shows a complete OpenClaw AgentSkill that uses the WorldState library to notarize agent decisions:

```
// SKILL.md frontmatter (YAML):
// ---
```

```
// name: ded-worldstate
// description: Notarize agent decisions to DED
// version: 1.0.0
// metadata:
//   openclaw:
//     emoji: "    "
//     bins: [node]
//     env: [DED_API_KEY, DED_PRIVATE_KEY, DED_ORG_ID]
//     os: [darwin, linux]
// ---

import { WorldState } from 'ded-agent-worldstate';
import { sha256, canonicalize } from '@constellation-network/metagraph-sdk';

const ws = await WorldState.create({
  privateKey: process.env.DED_PRIVATE_KEY!,
  orgId: process.env.DED_ORG_ID!,
  baseUrl: 'https://ded-api.constellationnetwork.io',
  apiKey: process.env.DED_API_KEY!,
  storagePath: `${process.env.HOME}/.openclaw/worldstate`,
});

// Create a stream for this agent's decision log
const decisions = ws.stream({
  tenantId: 'my-tenant-uuid',
  streamId: 'decisions-stream-uuid',
  appId: 'openclaw-skill-uuid',
});

// Skill handler: notarize a decision
export async function notarizeDecision(context: {
  action: string;
  reasoning: string;
  confidence: number;
  inputs: Record<string, unknown>;
}) {
  const stateHash = sha256(canonicalize(context));
  const eventId = crypto.randomUUID();

  const result = await decisions.insert(eventId, stateHash);

  return {
    message: `Decision notarized at ordinal ${result.ordinal}`,
    fingerprintHash: result.fingerprintHash,
    mptRoot: result.mptRoot,
    eventId,
  };
}

// Skill handler: verify a past decision
export async function verifyDecision(eventId: string) {
  const proof = await decisions.prove(eventId);
  const valid = await decisions.verify(proof);

  return {
    verified: valid,
    proof: proof.clientProof,
    fingerprintHash: proof.fingerprintHash,
  };
}
```

11 Cost Analysis

Operation	Credits	Cost (USD)	Notes
Fingerprint write	2	\$0.02	Per state insertion
Authenticated read	1	\$0.01	Filtered search, tags returned
Public read	0	Free	Fingerprint value only
Storage write	1/100 KB	\$0.01/100 KB	For checkpoint uploads
Per insertion	**2**	**\$0.02**	Fingerprint only

Table 1: Per-operation cost breakdown. Credits are \$0.01 each. Checkpoints are separate.

Scenario	Events/day	Daily Cost	Monthly Cost	Checkpoint
Low-frequency agent	10	\$0.20	\$6	\$0.22/wk
Moderate agent	100	\$2.00	\$60	\$0.22/day
High-frequency agent	1,000	\$20.00	\$600	\$0.22/day

Table 2: Projected costs by agent activity level. Checkpoint cost assumes ∼2 MB snapshot (∼10K entries) = 20 storage credits + 2 fingerprint credits.

Checkpoint cost: A 10,000-entry trie serializes to ∼2 MB. At 1 credit per 100 KB, a checkpoint costs ∼22 credits (\$0.22). Checkpointing every 1,000 insertions or daily keeps recovery cost predictable. Recovery requires downloading the latest checkpoint (1 read credit + storage download) and replaying only the entries since that checkpoint.

12 Threat Model

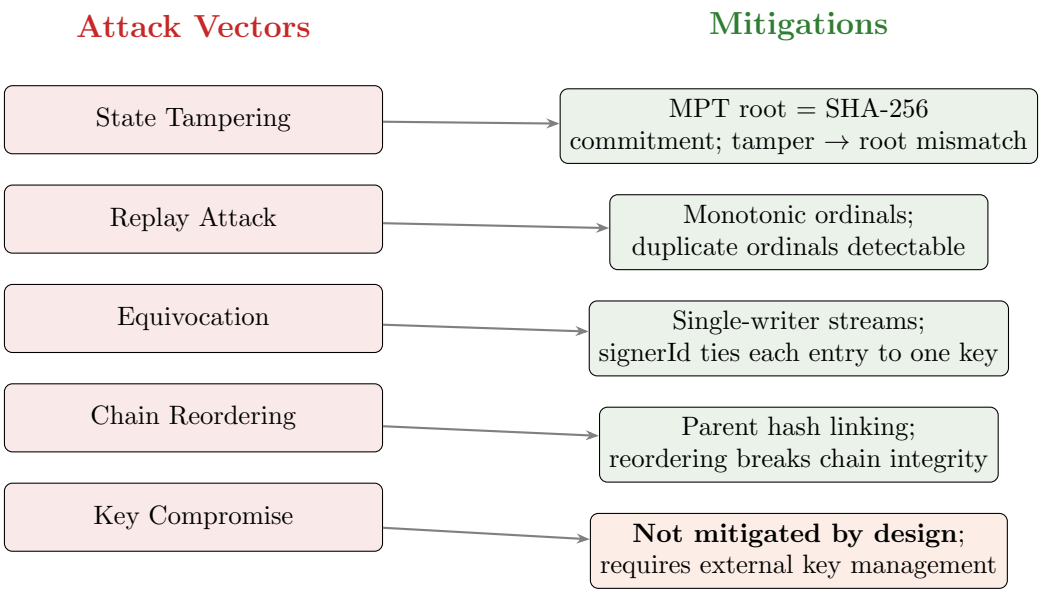


Figure 6: Threat model. The system defends against state tampering, replay, equivocation, and reordering. Key compromise is explicitly out of scope.

12.1 Security Properties

Tamper Evidence
Any modification to a past state entry changes the MPT root, which is committed as `documentRef` in an immutable fingerprint. The on-chain commitment makes retroactive modification detectable.

Non-Repudiation
Each fingerprint carries an ECDSA signature from the agent's key (`signerId`). The agent cannot deny having submitted a state update without claiming key compromise.

Temporal Ordering
The `parent:{hash}:ordinal:{N}` chain enforces a total order on events within a stream. The DED server enforces monotonic timestamps per organization.

Inclusion Provability
Any event's presence in the world state can be proven via MPT inclusion proof without revealing other entries in the trie.

12.2 Explicit Non-Goals

- **Confidentiality**: The system commits hashes, not plaintext. However, metadata (event IDs, timestamps, ordinals) is visible. If metadata confidentiality is required, additional encryption layers are needed.
- **Availability**: The system depends on the DED API and Constellation Network. Network outages prevent new commits but do not affect local state.
- **Key Management**: Secure storage and rotation of the agent's private key is the operator's responsibility.

13 Vision: Agent-to-Agent Trust

While the initial design focuses on *self-verification* — an agent proving the integrity of its own state — the architecture naturally extends to *inter-agent trust*:

1. **Shared Trust Layer**: The DED serves as a neutral, publicly queryable commitment log. Agent A publishes its world state roots; Agent B can verify A's claims by checking the on-chain commitments without direct communication.
2. **Cross-Agent Proofs**: If Agent A shares an MPT inclusion proof with Agent B (e.g., via DED Storage payloads), B can verify the proof against A's published `documentRef` without needing A's full trie.
3. **Trust Chains**: Agent A can include Agent B's latest fingerprint hash in its own state entries, creating a cross-reference. This enables verifiable dependency chains across agents: "A's decision at ordinal 5 was based on B's state at ordinal 12."
4. **Reputation via Commitment History**: An agent's commitment history — frequency, consistency, chain integrity — becomes a verifiable reputation signal. Agents with long, unbroken commitment chains demonstrate operational reliability.

This vision positions the DED not merely as a notarization service, but as a **decentralized trust substrate for autonomous agent ecosystems**.

14 Implementation Roadmap

Phase	Deliverable	Description
1	MPT Port (TypeScript)	Port `MerklePatriciaTrie`, `Producer`, `Prover`, `Verifier` from metakit Scala to `@constellation-network/metagraph-sdk` TypeScript. Include shared test vectors for cross-language compatibility.
2	FingerprintValue Extension	Add `stream_id` and `app_id` to proto schema, ded-sdk types, and validation. Backwards-compatible (optional with defaults).
3	`ded-agent-worldstate`	Core library: `WorldState`, per-stream MPT management, chain linking, LevelDB persistence, DED API integration.
4	Checkpoint & Recovery	DED Storage integration for periodic trie snapshot uploads. Recovery procedure: checkpoint download + incremental replay.
5	OpenClaw Skill Example	Reference OpenClaw AgentSkill demonstrating end-to-end integration. Published to ClawHub.

Table 3: Implementation phases. Each phase builds on the previous and is independently testable.

Appendix A: Server-Side Alignment

The following server-side changes are required to fully support the extended schema and stream-aware features. These are documented here for the DED server team and are **not** part of the client-side implementation scope.

A.1 Protobuf Schema

Add `stream_id` (field 9) and `app_id` (field 10) to `DigitalEvidence_v1.proto` as optional string fields with UUID validation rules.

A.2 Database Migration

```
-- V<N>__add_stream_app_fields.sql
ALTER TABLE fingerprint_values
  ADD COLUMN stream_id UUID,
  ADD COLUMN app_id UUID;

CREATE INDEX idx_fpv_stream ON fingerprint_values (tenant_id, stream_id,
    app_id);
```

A.3 PathEvidenceCalculator

Update to read `stream_id` and `app_id` from the fingerprint value instead of defaulting to the zero UUID. Fallback to zero UUID when fields are absent (backwards compatibility).

A.4 Stream-Aware Batching

Modify the `BatchDaemon` to prefer grouping fingerprints from the same `(tenant_id, stream_id, app_id)` into the same batch when possible. This preserves chain ordering in the server-side MPT and simplifies proof generation.

A.5 API Key Scope

No changes needed. `stream_id` and `app_id` are free-form per tenant — any tenant can create arbitrary stream/app identifiers within their organization.

A.6 Query API

Add `stream_id` and `app_id` as optional query parameters to `GET /v1/fingerprints` for filtered searches.

Appendix B: MPT Hashing Specification

The MPT uses SHA-256 with type-prefix bytes to compute node digests. This matches the metakit Scala implementation:

$$\text{LeafDigest} = \text{SHA-256}(\texttt{0x00} \| \text{canonical}(\{\texttt{remaining} : \text{nibbles}, \texttt{dataDigest} : \text{SHA-256}(\text{data})\}))$$

$$\text{BranchDigest} = \text{SHA-256}(\texttt{0x01} \| \text{canonical}(\{\texttt{pathsDigest} : \{n_0 : d_0, n_1 : d_1, \ldots\}\}))$$

$$\text{ExtensionDigest} = \text{SHA-256}(\texttt{0x02} \| \text{canonical}(\{\texttt{shared} : \text{nibbles}, \texttt{childDigest} : d_{\text{branch}}\}))$$

Where canonical($\cdot$) is RFC 8785 JSON Canonicalization Scheme applied to the commitment object, then UTF-8 encoded to bytes.